

Principal_Component_Analysis

September 6, 2026

Principal Component Analysis

0.1 ### Task 2: Load the Data and Libraries

```
[1]: %matplotlib inline
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

```
[2]: plt.style.use("ggplot")
plt.rcParams["figure.figsize"] = (12,8)
```

```
[3]: # data URL: https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.
      ↪data

iris = pd.read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/
      ↪iris/iris.data", header=None)
iris.head()
```

```
[3]:      0      1      2      3      4
0  5.1  3.5  1.4  0.2  Iris-setosa
1  4.9  3.0  1.4  0.2  Iris-setosa
2  4.7  3.2  1.3  0.2  Iris-setosa
3  4.6  3.1  1.5  0.2  Iris-setosa
4  5.0  3.6  1.4  0.2  Iris-setosa
```

```
[6]: iris.columns = ["sepal_length", "sepal_width", "petal_length", "petal_width",
      ↪"species"]
iris.dropna(how='all', inplace=True)
iris.head()
```

```
[6]:      sepal_length  sepal_width  petal_length  petal_width  species
0              5.1           3.5           1.4           0.2  Iris-setosa
1              4.9           3.0           1.4           0.2  Iris-setosa
2              4.7           3.2           1.3           0.2  Iris-setosa
3              4.6           3.1           1.5           0.2  Iris-setosa
4              5.0           3.6           1.4           0.2  Iris-setosa
```

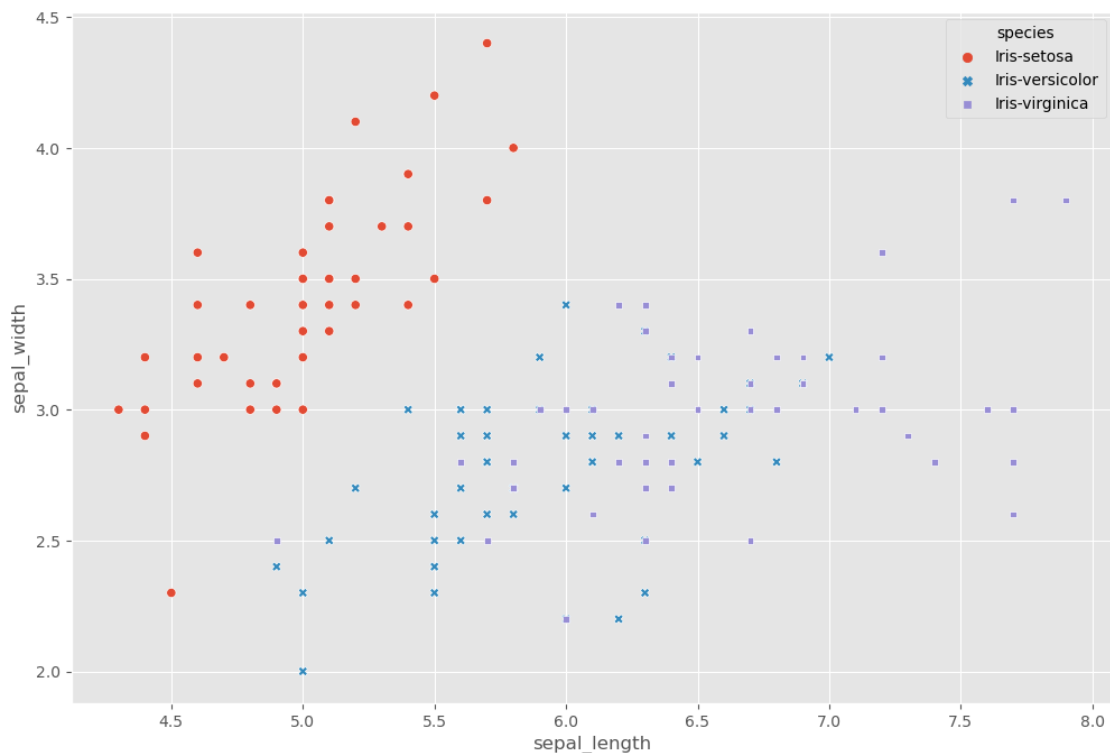
```
[7]: iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 150 entries, 0 to 149  
Data columns (total 5 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   sepal_length    150 non-null   float64  
1   sepal_width     150 non-null   float64  
2   petal_length    150 non-null   float64  
3   petal_width     150 non-null   float64  
4   species         150 non-null   object  
dtypes: float64(4), object(1)  
memory usage: 6.0+ KB
```

0.2 ### Task 3: Visualize the Data

```
[10]: sns.scatterplot(x=iris.sepal_length, y=iris.sepal_width, hue=iris.species,   
↪ style=iris.species)
```

```
[10]: <AxesSubplot: xlabel='sepal_length', ylabel='sepal_width'>
```



0.3 ### Task 4: Standardize the Data

```
[15]: x = iris.iloc[:,0:4].values
      y = iris.species.values
```

```
[16]: from sklearn.preprocessing import StandardScaler

      X = StandardScaler().fit_transform(x)
```

0.4 ### Task 5: Compute the Eigenvectors and Eigenvalues

Covariance: $\sigma_{jk} = \frac{1}{n-1} \sum_{i=1}^N (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$

Covariance matrix: $\Sigma = \frac{1}{n-1} ((X - \bar{x})^T (X - \bar{x}))$

```
[20]: covariance_matrix = np.cov(X.T)

      print("Covariance matrix: \n", covariance_matrix)
```

Covariance matrix:

```
[[ 1.00671141 -0.11010327  0.87760486  0.82344326]
 [-0.11010327  1.00671141 -0.42333835 -0.358937  ]
 [ 0.87760486 -0.42333835  1.00671141  0.96921855]
 [ 0.82344326 -0.358937    0.96921855  1.00671141]]
```

We can prove this by looking at the covariance matrix. It has the property that it is symmetric. We also constrain the each of the columns (eigenvectors) such that the values sum to one. Thus, they are orthonormal to each other.

Eigendecomposition of the covariance matrix: $\Sigma = W \Lambda W^{-1}$

```
[21]: eigen_values, eigen_vectors = np.linalg.eig(covariance_matrix)
      print("Eigen vectors: \n", eigen_vectors, "\n")
      print("Eigen vals: \n", eigen_values)
```

Eigen vectors:

```
[[ 0.52237162 -0.37231836 -0.72101681  0.26199559]
 [-0.26335492 -0.92555649  0.24203288 -0.12413481]
 [ 0.58125401 -0.02109478  0.14089226 -0.80115427]
 [ 0.56561105 -0.06541577  0.6338014   0.52354627]]
```

Eigen vals:

```
[2.93035378 0.92740362 0.14834223 0.02074601]
```

0.5 ### Task 6: Singular Value Decomposition (SVD)

```
[22]: eigen_vec_svd, s, v = np.linalg.svd(X.T)
      eigen_vec_svd
```

```
[22]: array([[ -0.52237162, -0.37231836,  0.72101681,  0.26199559],
            [ 0.26335492, -0.92555649, -0.24203288, -0.12413481],
            [-0.58125401, -0.02109478, -0.14089226, -0.80115427],
            [-0.56561105, -0.06541577, -0.6338014 ,  0.52354627]])
```

```
[ ]:
```

0.6 ### Task 7: Picking Principal Components Using the Explained Variance

```
[23]: for val in eigen_values:
        print(val)
```

```
2.930353775589317
0.9274036215173428
0.14834222648163967
0.020746013995595967
```

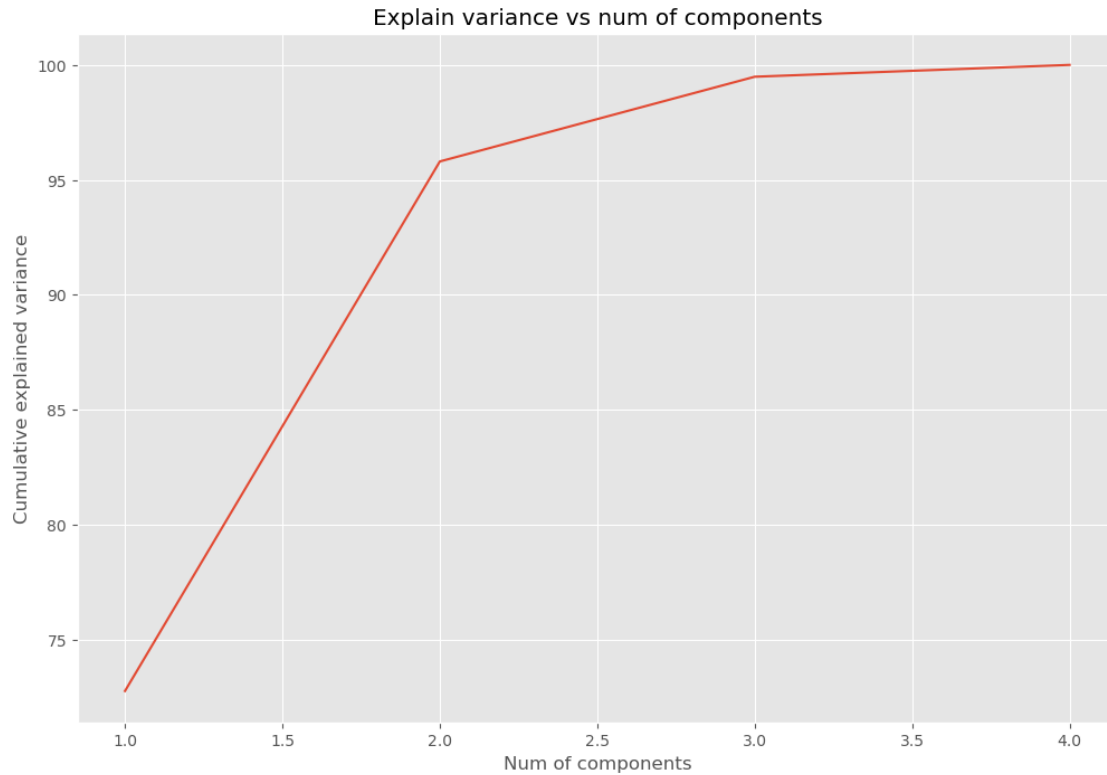
```
[25]: variance_explained = [(i/sum(eigen_values))*100 for i in eigen_values]
        variance_explained
```

```
[25]: [72.77045209380132, 23.03052326768066, 3.6838319576273824, 0.5151926808906327]
```

```
[26]: cumulative_variance_explained = np.cumsum(variance_explained)
        cumulative_variance_explained
```

```
[26]: array([ 72.77045209,  95.80097536,  99.48480732, 100.          ])
```

```
[27]: sns.lineplot(x=[1,2,3,4], y=cumulative_variance_explained)
        plt.xlabel("Num of components")
        plt.ylabel("Cumulative explained variance")
        plt.title("Explain variance vs num of components")
        plt.show()
```



0.7 ### Task 8: Project Data Onto Lower-Dimensional Linear Subspace

```
[28]: eigen_vectors
```

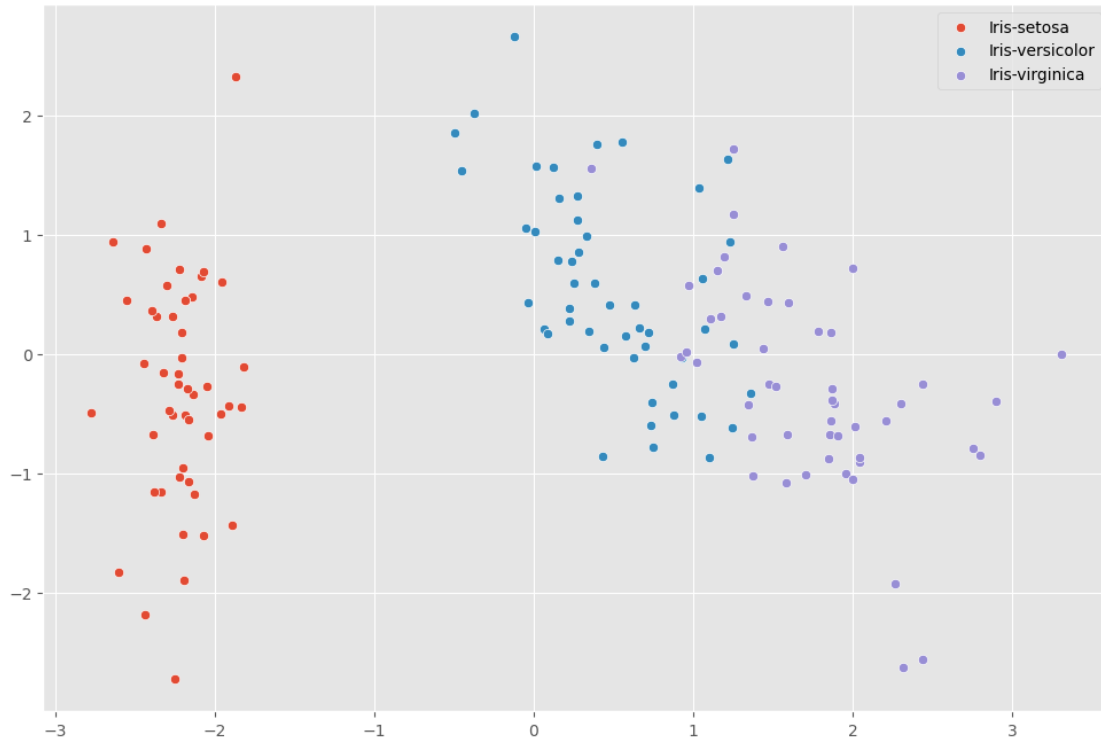
```
[28]: array([[ 0.52237162, -0.37231836, -0.72101681,  0.26199559],
             [-0.26335492, -0.92555649,  0.24203288, -0.12413481],
             [ 0.58125401, -0.02109478,  0.14089226, -0.80115427],
             [ 0.56561105, -0.06541577,  0.6338014 ,  0.52354627]])
```

```
[30]: projection_matrix = (eigen_vectors.T[:][:])[:2].T #choosing only 1st 2
      ↪ principal components
      print("Matrix : \n", projection_matrix)
```

```
Matrix :
[[ 0.52237162 -0.37231836]
 [-0.26335492 -0.92555649]
 [ 0.58125401 -0.02109478]
 [ 0.56561105 -0.06541577]]
```

```
[34]: X_pca = X.dot(projection_matrix) #X_pca = X.W
```

```
[36]: for species in ('Iris-setosa', 'Iris-versicolor', 'Iris-virginica'):
    sns.scatterplot(
        x=X_pca[y == species, 0],
        y=X_pca[y == species, 1],
        label=species
    )
```



We started off with the dataset, identified the feature variables and target classes, and then pre-processed the data. Initially, the classes were difficult to separate clearly in the original feature space.

To address this, we first **standardized the features** so that they were on a comparable scale. We then applied **Principal Component Analysis (PCA)** to reduce the dimensionality of the dataset.

In PCA, we derived the principal components by:

- Calculating the **covariance matrix** of the standardized data.
- Performing **eigendecomposition** to obtain the **eigenvalues and eigenvectors**.
- Alternatively, PCA can be computed using **Singular Value Decomposition (SVD)**.

The **eigenvalues** indicate how much variance each principal component explains. We used the **cumulative explained variance** to determine how many components to retain. By setting a threshold of **95% explained variance**, we found that the **first two principal components** captured approximately 95% of the information in the dataset.

Therefore, we selected the first two eigenvectors to form the **projection matrix**. We then projected the original standardized data onto this lower-dimensional space by taking the dot product of the data matrix with the projection matrix.

As a result, the original dataset was reduced from **4 dimensions (features) to 2 dimensions**, while retaining approximately **95% of the dataset's variance**. This lower-dimensional representation makes it easier to visualize and analyze the separation between the different classes.